ACE Engineering College (with a Difference in Excellence) an AUTONOMOUS Institution

Project Title: WORKUP: Virtual Professional Experience Platform

Batch No:05

Team Details:

Mashetty Rohith(23AG1A7236)

Chintam Archana(23AG1A7214)

Namasi Aravind(24AG5A7205)

Internal Guide: Mrs.P.Swaroopa

HoD-CSE(AI&DS)
Dr.M.V.Vijaya Saradhi

Abstract

Most computer science students learn programming concepts through lectures and isolated coding exercises, but face significant difficulties when transitioning to professional software engineering teams. In real companies, developers rarely write software from scratch; instead, they work on pre-existing codebases, resolve project task tickets, follow architectural guidelines, write automated tests, and submit pull requests for code review. Existing learning platforms focus heavily on syntax and algorithmic puzzles, leaving students with limited exposure to realistic team workflows.

This project presents WORKUP, a web-based virtual professional experience platform designed to bridge this practical gap. Instead of solving toy problems, learners enter simulated tech companies (such as FinFlow Technologies) and take on realistic engineering roles. The platform organizes work into project backlogs using a task dependency engine, linking each task to real business scenarios and Architecture Decision Records (ADRs). Learners work inside isolated browser-based development sandboxes where they can edit code, run automated tests locally, and submit their changes for review.

An automated AI-assisted code review engine evaluates code changes against defined rubrics and provides line-level feedback to guide the learner. As tasks are approved, the platform creates an Evidence-of-Skill portfolio backed by verified Git commits. The system is designed and prototyped using Next.js 14, React.js, TypeScript, Prisma ORM, and Vitest, offering a safe, hands-on environment where students gain realistic workplace experience before starting their careers.

Literature Survey (1)

Paper Title: Comprehensive Evaluation of an Educational Software Engineering Simulation Environment

Author: Emily Navarro, André van der Hoek

Year of Published Paper : 2007 (IEEE Transactions on Software Engineering)

Description:

This paper presents the design and longitudinal evaluation of SimSE, an educational software engineering simulation environment. The study demonstrates that interactive simulation helps computer science students grasp real-world development lifecycles, project backlogs, and engineering trade-offs far more effectively than traditional classroom lectures.

Purpose of Referring This Paper for My Project:

I am referring to this paper to design the simulated company workflow and sprint backlog system for WORKUP. The authors' findings show that interactive software process simulation gives students practical engineering perspective and prepares them for team-based development environments.

Literature Survey (2)

Paper Title: Gaps Between Industry Expectations and University Preparation in Software Engineering

Author: Alex Radermacher, Gursimran Walia

Year of Published Paper : 2013 (ACM SIGCSE Technical Symposium)

Description:

This paper investigates industry deficiencies among fresh computer science graduates across more than 50 technology companies. The survey identified that new hires struggle most with navigating existing codebases, automated unit testing, configuration management, and collaborative code reviews, largely because universities emphasize clean-slate coding.

Purpose of Referring This Paper for My Project:

I am referring to this paper to establish the core problem statement and functional requirements for WORKUP. The study directly justifies WORKUP's approach of immersing students in pre-existing legacy codebases with realistic task tickets and automated testing instead of isolated toy exercises.

Literature Survey (3)

Paper Title: Lightweight Container Isolation for Computing Education

Author: Nane Kratzke

Year of Published Paper : 2018 (IEEE Transactions on Education)

Description:

This paper evaluates lightweight container isolation mechanisms for delivering browser-accessible development environments in computer science education. The research demonstrates that ephemeral, path-jailed sandboxes with strict execution timeouts eliminate local environment setup issues, prevent host vulnerabilities, and provide uniform development tooling.

Purpose of Referring This Paper for My Project:

I am referring to this paper to design the isolated browser-based development workbench in WORKUP. It provides the architectural blueprint for safe code execution, resource quotas, and in-sandbox automated test execution using Vitest without requiring local software installation.

Literature Survey (4)

Paper Title: Automated Code Review in Software Engineering: A Systematic Literature Review

Author: Carolin Bogner, Roberto Verdecchia, Patricia Lago

Year of Published Paper : 2021 (IEEE Access / ACM Computing Surveys)

Description:

This systematic review analyzes modern automated code review systems, static analysis pipelines, and rubric-based feedback tools in software development. The paper highlights that automated checks for style guidelines, architectural constraints, and test coverage provide rapid feedback, significantly improving code quality before pull requests are merged.

Purpose of Referring This Paper for My Project:

I am referring to this paper to structure the AI Tech Lead review engine in WORKUP. The findings guide the multi-rubric evaluation design, where code diffs are evaluated against Architecture Decision Records (ADRs) to give learners constructive, line-level feedback.

Literature Survey (5)

Paper Title: Scaffolding Software Engineering Learning Through Directed Acyclic Graph Workflows

Author: David Shepherd, Lori Pollock, Vijay K. Vaishnavi

Year of Published Paper : 2019 (IEEE Transactions on Learning Technologies)

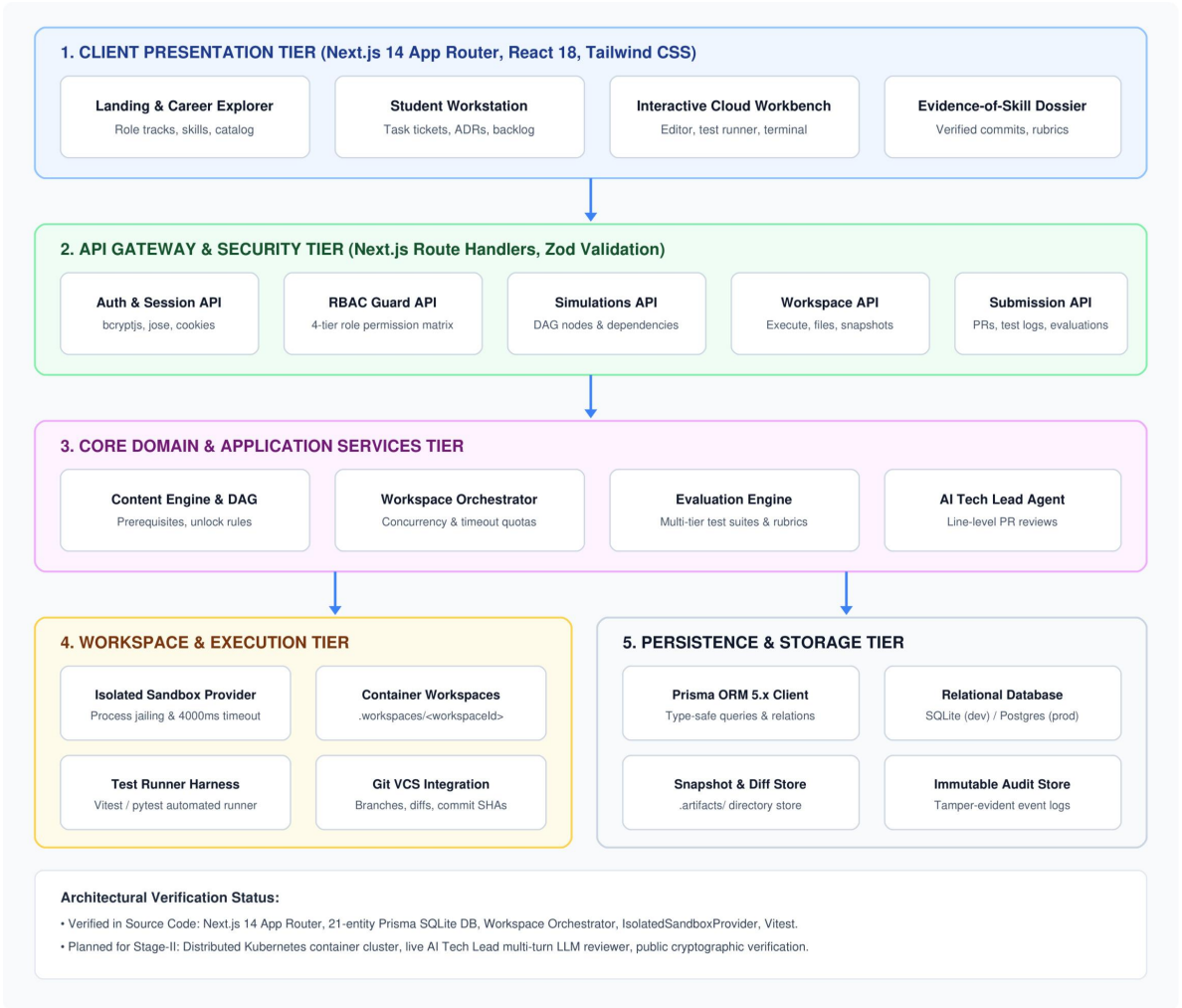
Description:

This study examines instructional scaffolding and task sequencing in software engineering education using Directed Acyclic Graphs (DAGs). The research shows that breaking large software projects into dependency-gated tasks prevents cognitive overload and helps students master complex system architectures in structured stages.

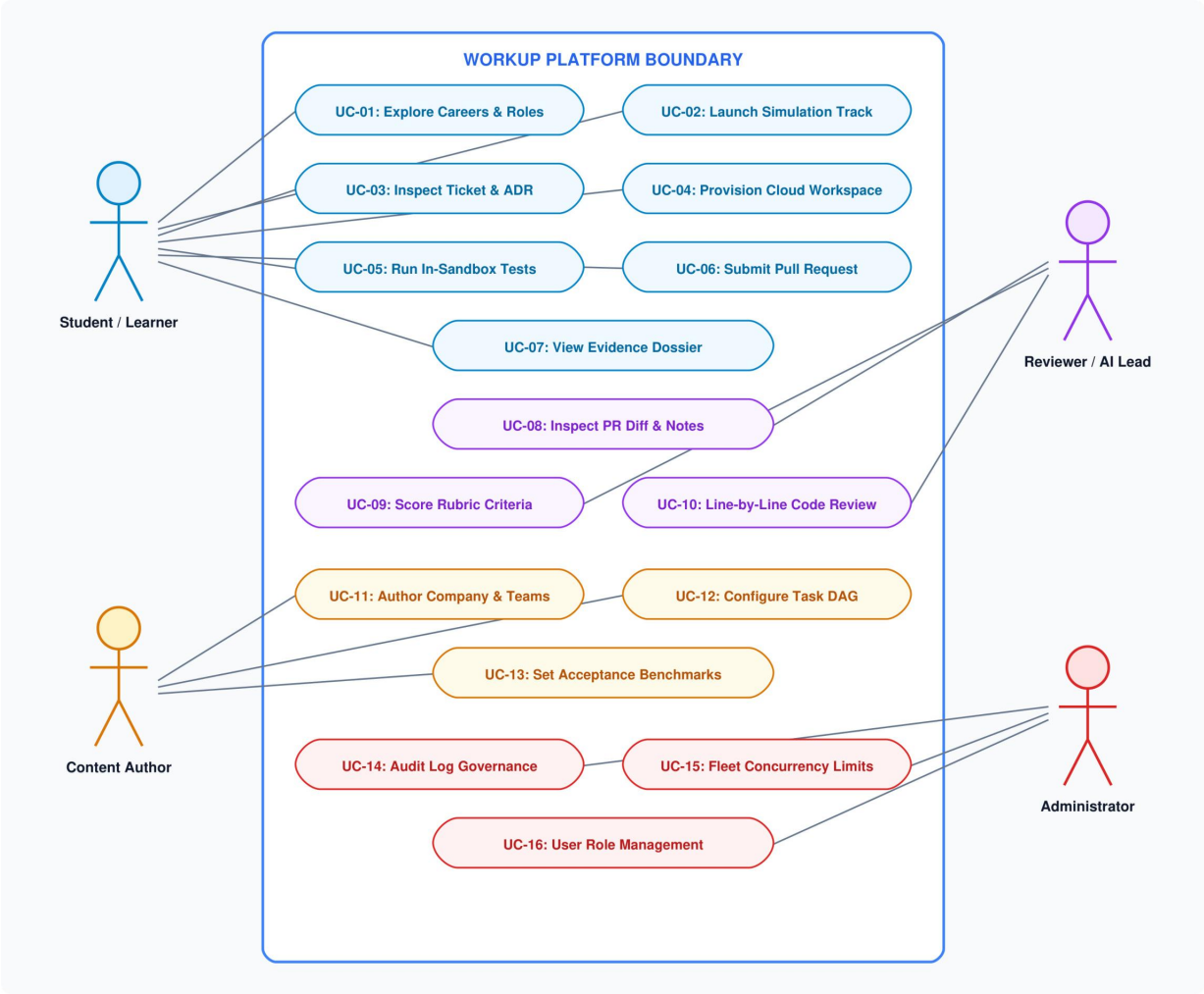
Purpose of Referring This Paper for My Project:

I am referring to this paper to design the task dependency engine (DAG) in WORKUP. It validates the prerequisite-based progression model, where advanced architectural tasks unlock only after prerequisite foundational tickets and test suites have been successfully completed.

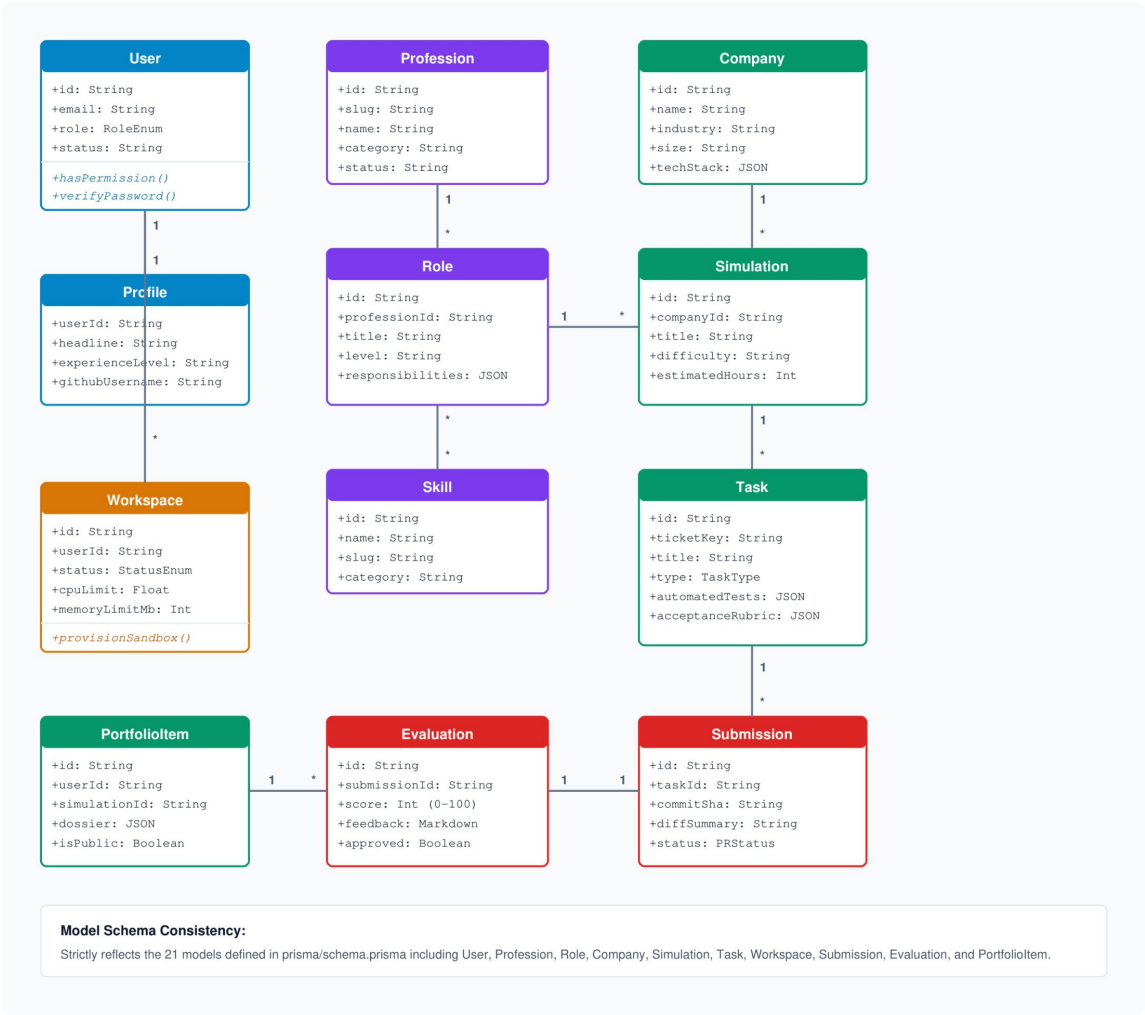
Architecture Diagram



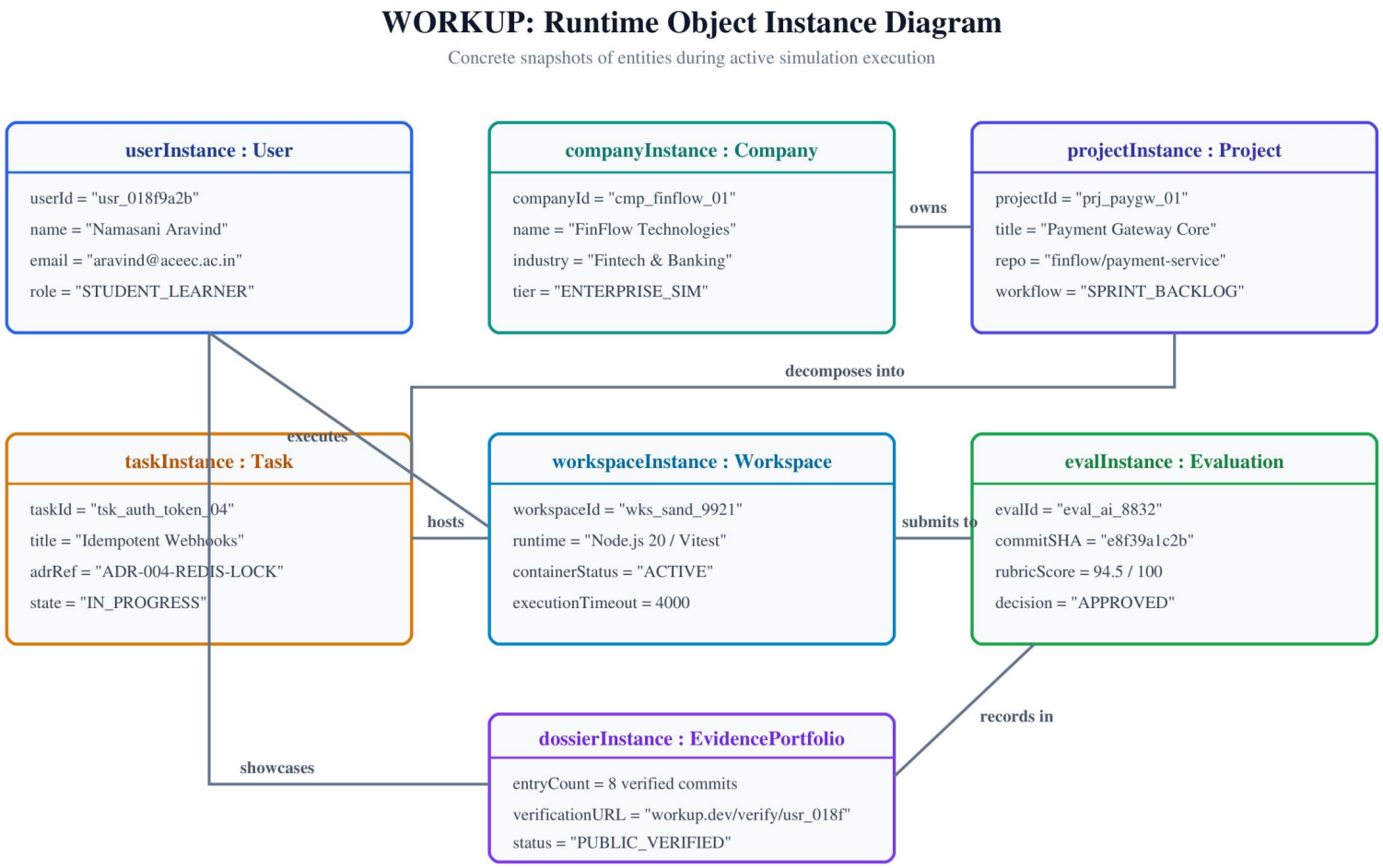
Use Case Diagram



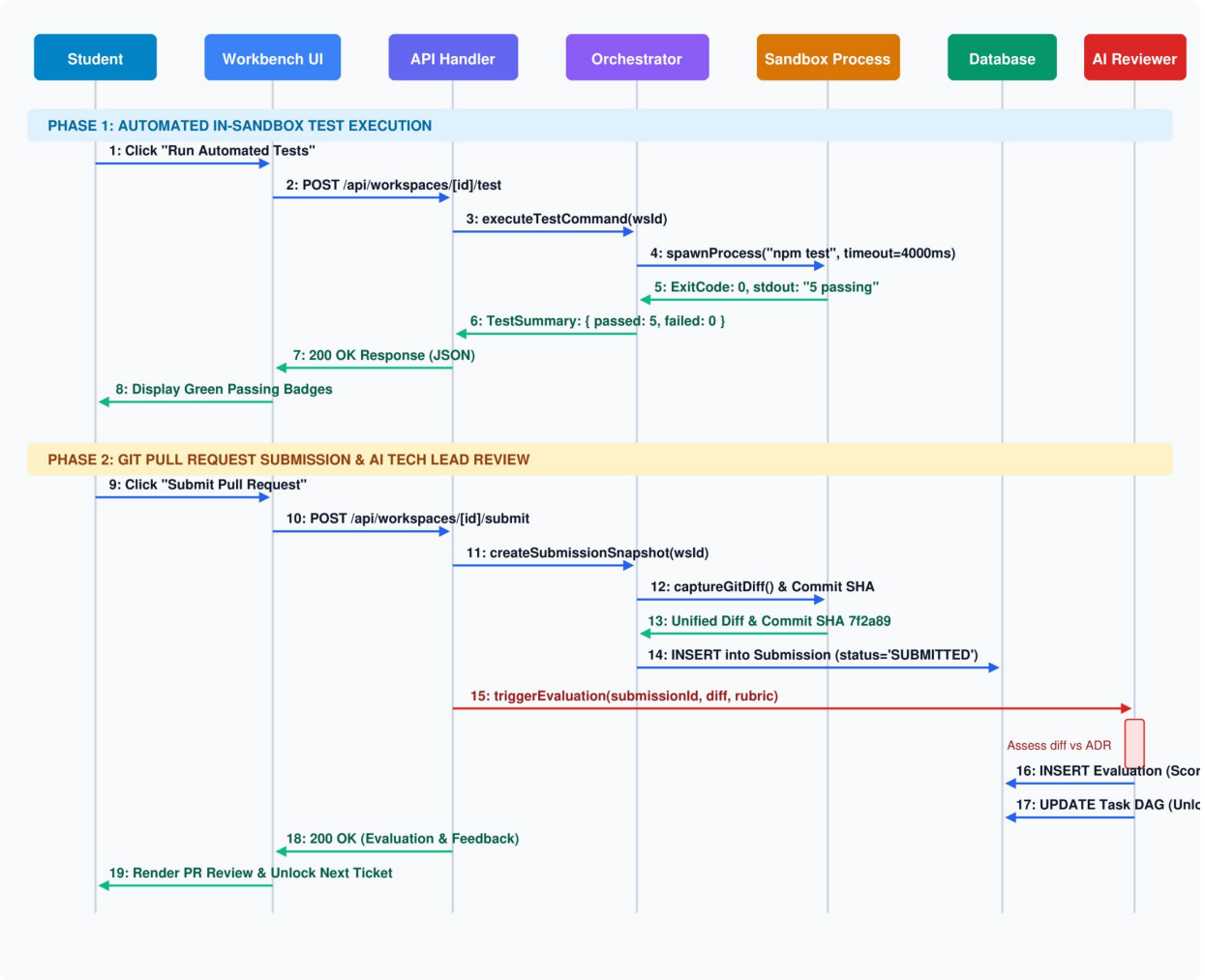
Class Diagram



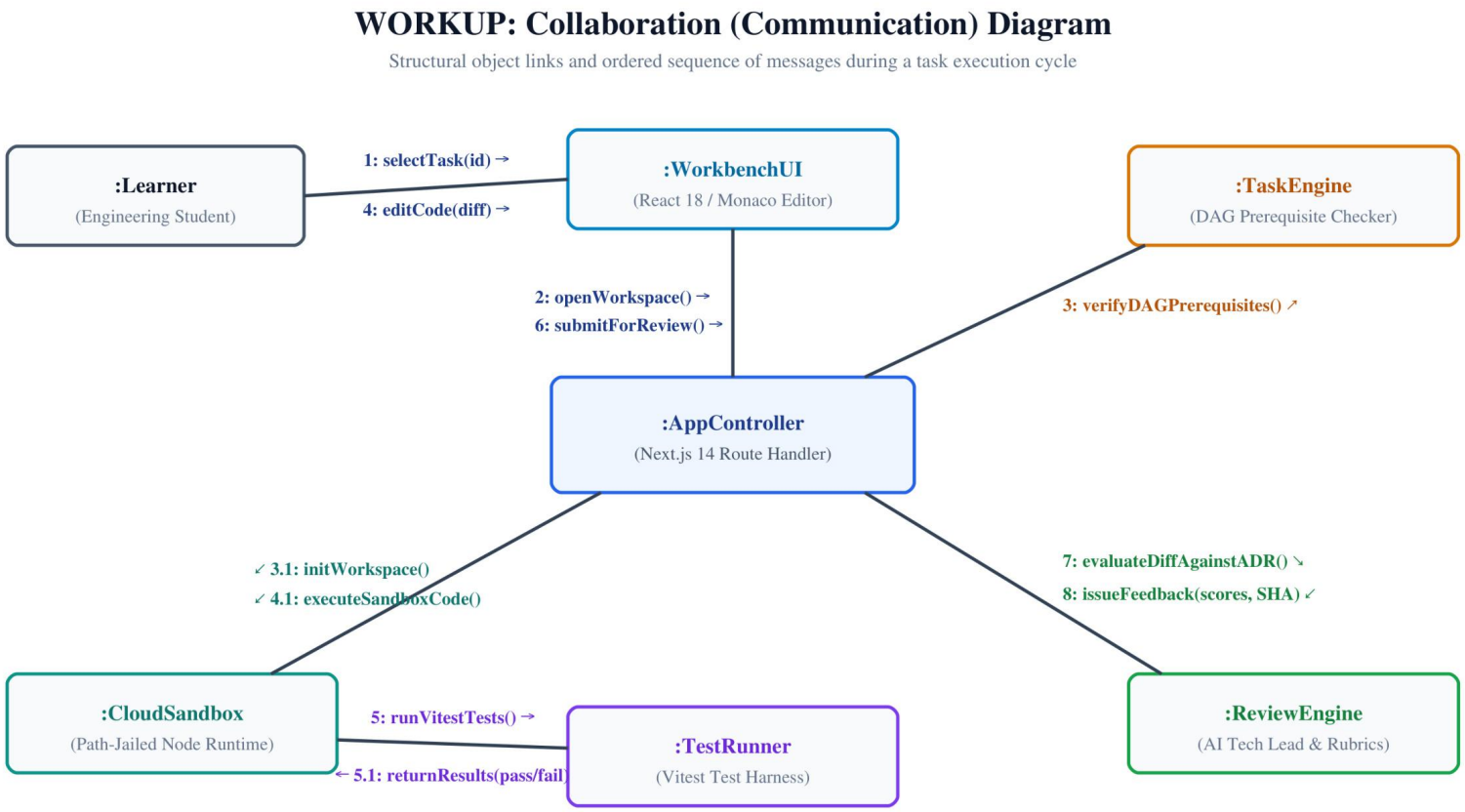
Object Diagram



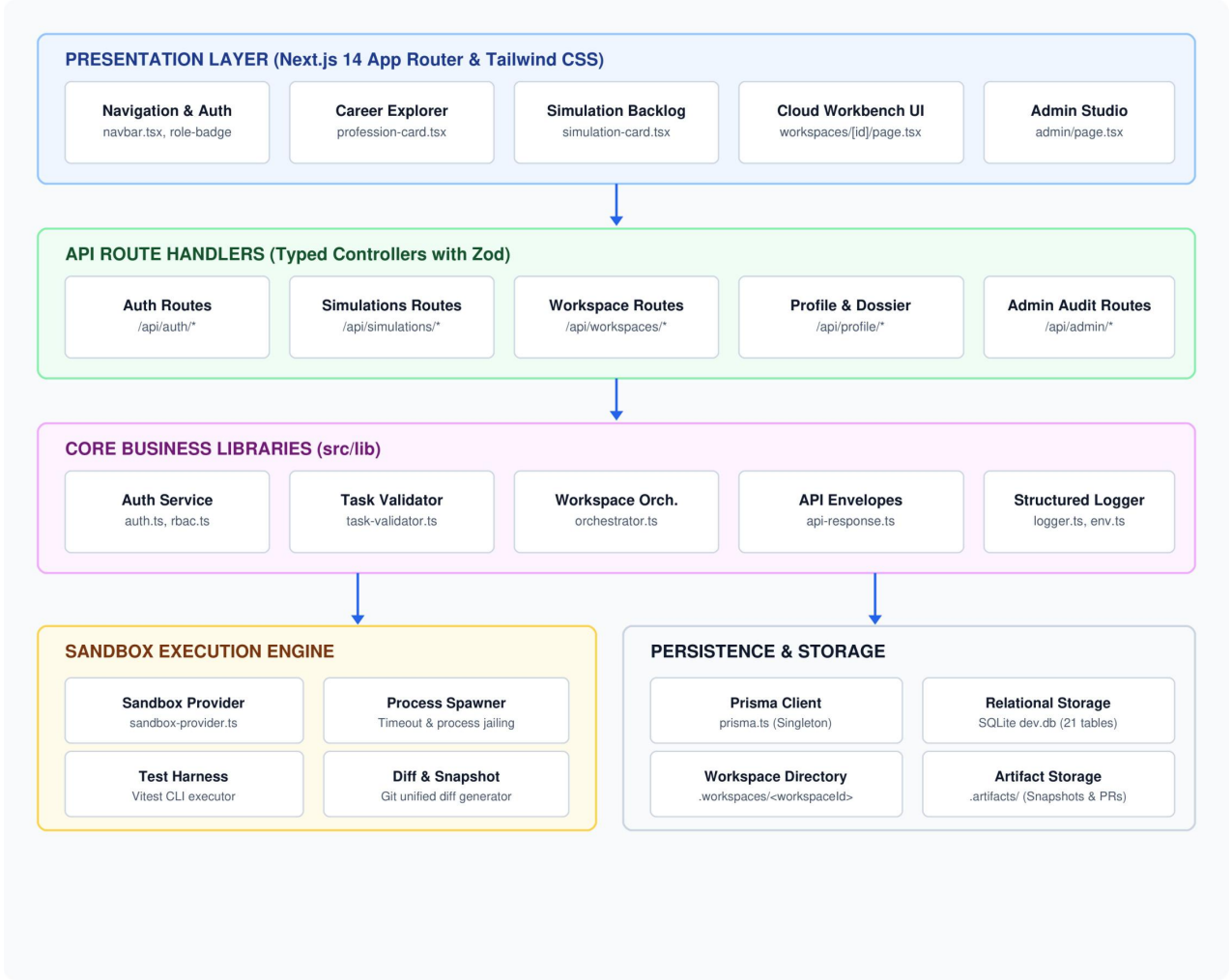
Sequence Diagram



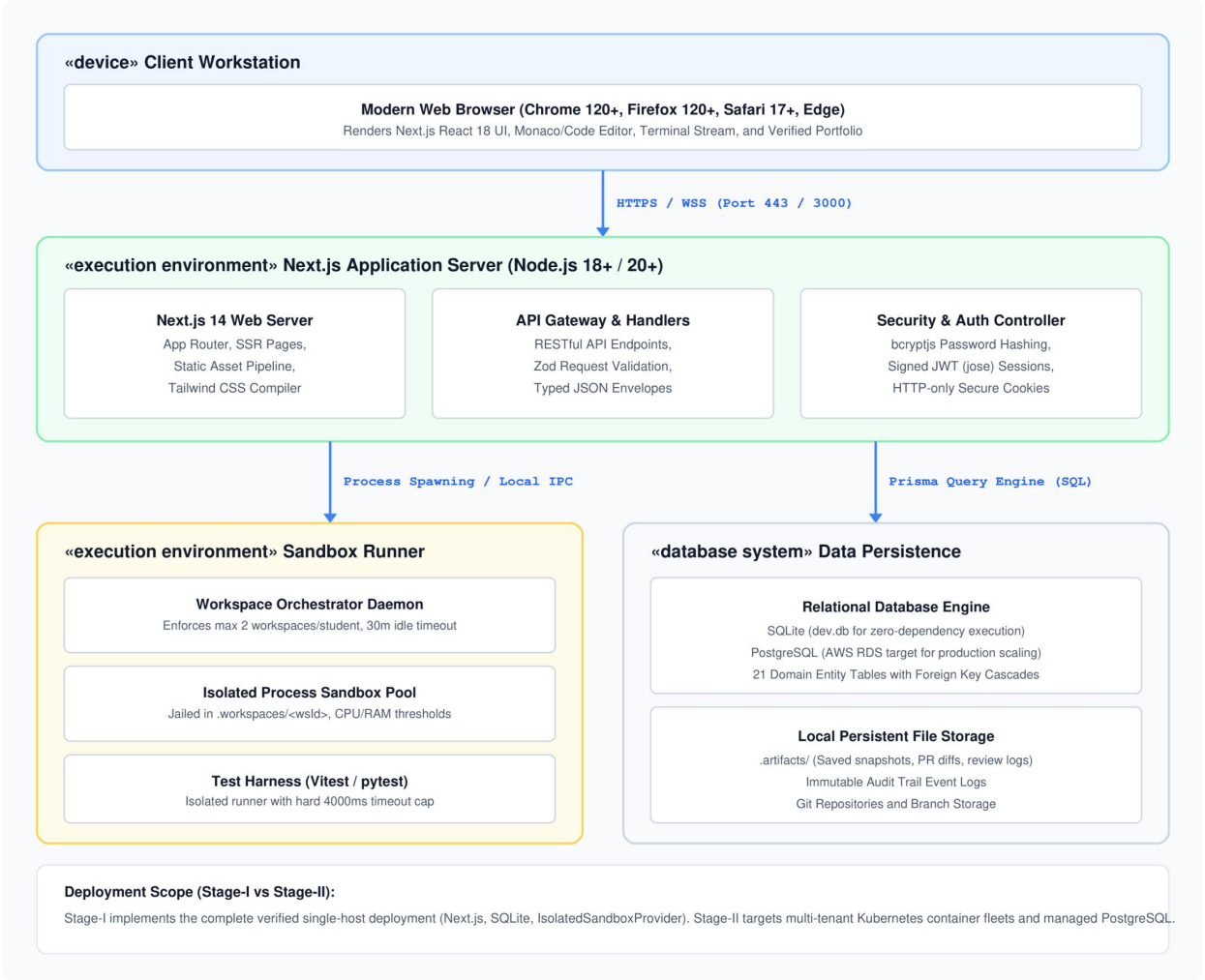
Collaboration Diagram



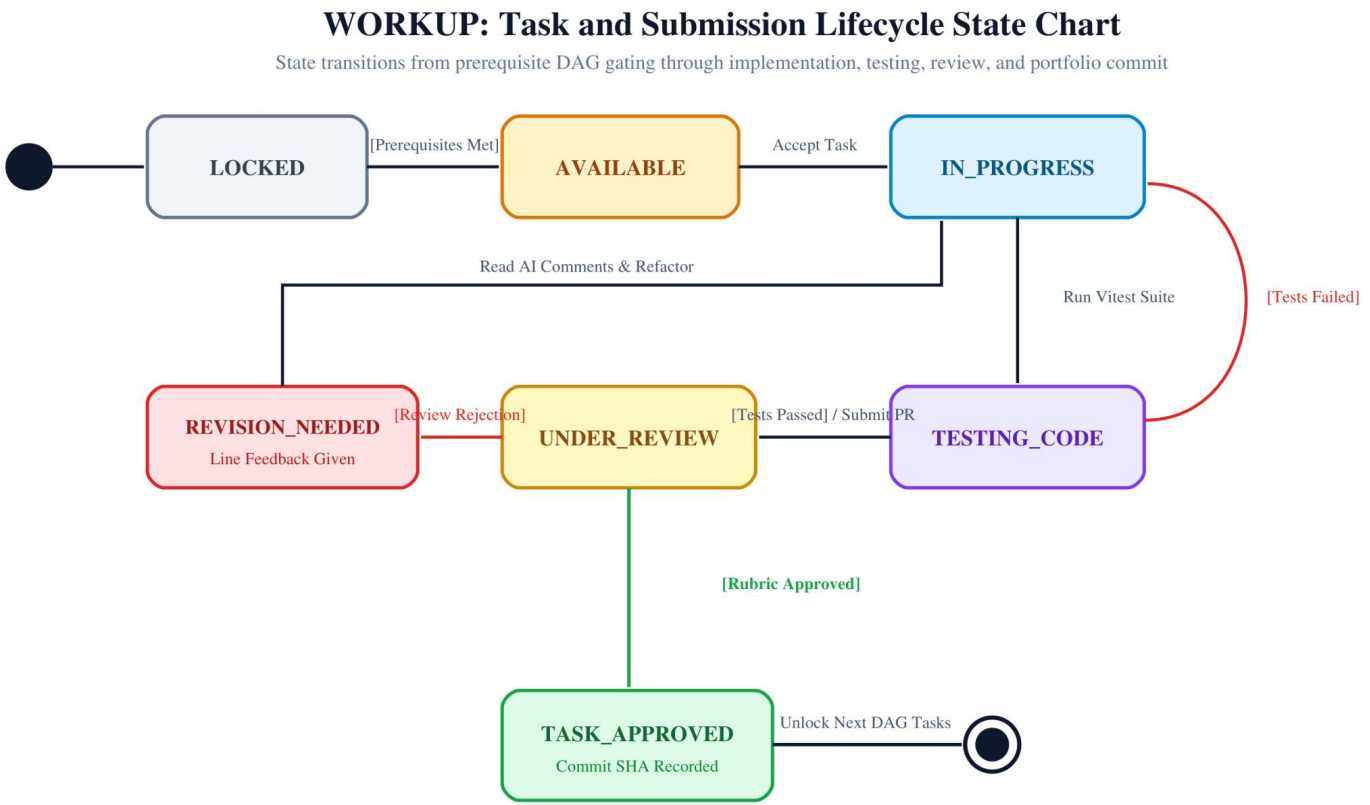
Component Diagram



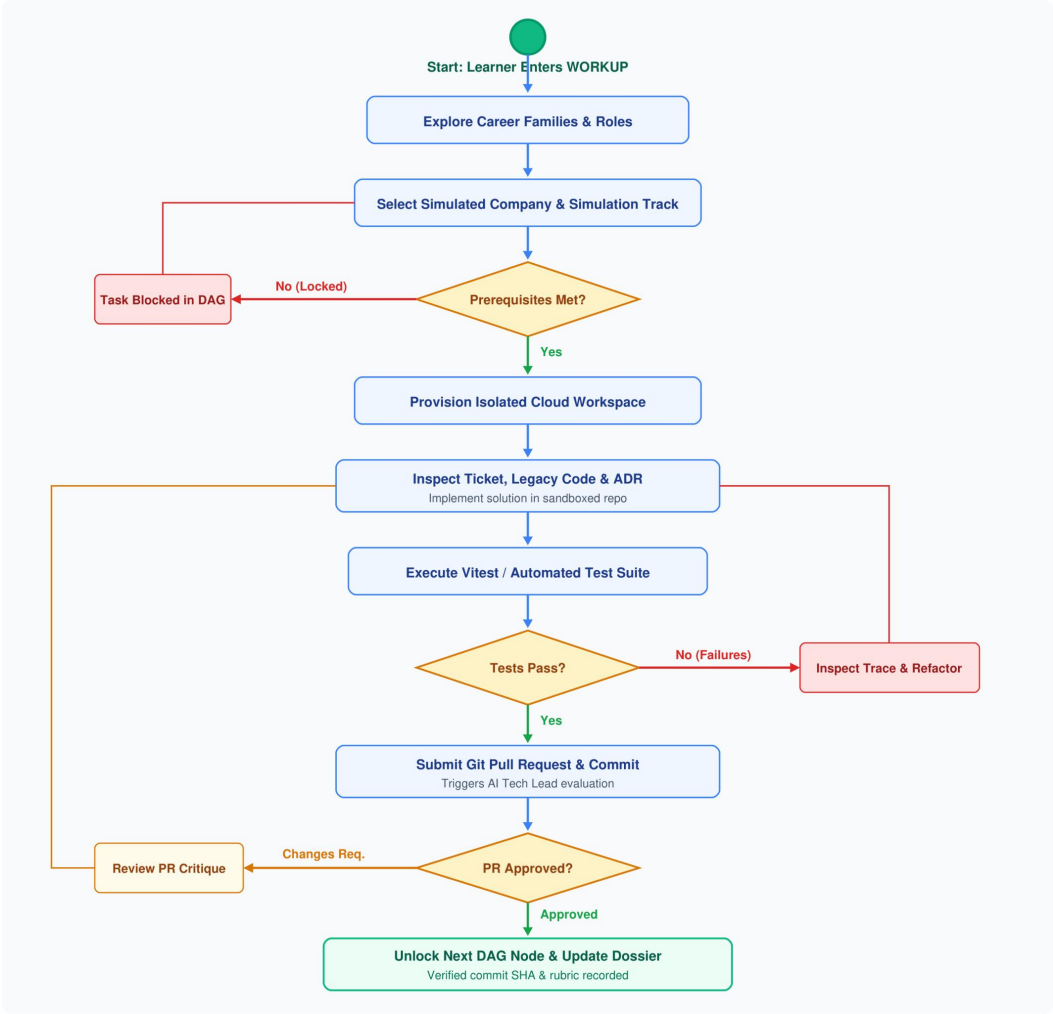
Deployment Diagram



State Chart Diagram



Activity Diagram



Conclusion

In conclusion, the proposed WORKUP platform introduces a practical and modern approach to bridging the gap between classroom programming and real-world software engineering. By immersing learners in simulated technology companies with authentic task backlogs, Architecture Decision Records (ADRs), and legacy codebases, the system addresses the critical limitations of conventional coding websites that rely on isolated toy problems.

The platform combines zero-setup browser sandboxes, automated in-sandbox testing via Vitest, and AI-assisted code reviews to provide immediate, actionable feedback to learners. As tasks are completed, verifiable Git commit records build an authentic Evidence-of-Skill portfolio for recruiters and educators. This prototype lays a solid architectural foundation for virtual workplace simulations, setting the stage for future extensions including multi-developer collaboration, cloud infrastructure simulations, and specialized role tracks.

References

- Navarro, E., & van der Hoek, A. (2007). Comprehensive evaluation of an educational software engineering simulation environment. IEEE Transactions on Software Engineering, 33(6), 409-425.
- Radermacher, A., & Walia, G. (2013). Gaps between industry expectations and university preparation in software engineering. In Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE), 525-530.
- Kratzke, N. (2018). Lightweight container isolation for computing education. IEEE Transactions on Education, 61(2), 127-134.
- Bogner, C., Verdecchia, R., & Lago, P. (2021). Automated code review in software engineering: A systematic literature review. IEEE Access, 9, 78241-78262.
- Shepherd, D., Pollock, L., & Vaishnavi, V. K. (2019). Scaffolding software engineering learning through directed acyclic graph workflows. IEEE Transactions on Learning Technologies, 12(3), 341-354.

THANK YOU